

NeetCode 150 User Guide

A practice log for learning algorithms by hand, with tooling that keeps you honest.

Contents

1. What this repo is	4
2. Repo tour	4
3. Where to work	5
4. The tools	5
Script: new_problem.py	6
Script: progress.py	6
Script: check_file.py	6
Script: flashcards.py	7
Script: install_hooks.py	7
The slash commands	7
The pre-commit hook	7
CI	8
5. The daily loop	8
6. The weekly loop	9
7. How to approach any problem	10
8. Category primers	10
Arrays and Hashing	11
Two Pointers	12
Sliding Window	13
Stack	14
Binary Search	15
Linked List	16
Trees	17
Tries	18
Heap / Priority Queue	19
Backtracking	20
Graphs	21
Advanced Graphs	22
1-D Dynamic Programming	23
2-D Dynamic Programming	24

Greedy	25
Intervals	26
Math and Geometry	27
Bit Manipulation	28
9. Getting unstuck	28
10. Troubleshooting	29

This guide explains the whole repo from first principles. It assumes you have never typed a slash command, do not yet know what the scripts do and are not sure whether to solve problems on a website or on your own computer. Read it once end to end, then keep it open as a reference.

1. What this repo is

This repo is a place to work through the NeetCode 150, a curated list of 150 coding interview problems grouped into 18 topics. It is a learning log, not a solutions bank. You write every line of every solution yourself. The tooling here scaffolds files, runs your own tests, tracks your progress and reminds you what to revisit. It never writes the answer for you.

There is one rule, and it does not bend: no AI-written solutions, ever. Not the full answer, not a partial answer, not pseudocode and not "just this one line". If you ask the assistant to solve a problem, it will refuse and offer a hint instead. The full rules live in the file `CLAUDE.md` at the top of the repo.

The rule exists because the point of this practice is the struggle. When you force yourself to work a problem out, you build the pattern recognition that an interview actually tests. If an assistant hands you the code, you get a working file and learn nothing. A tool that solves the problem for you is worse than no tool, because it feels like progress while removing the part that teaches you. So the tooling is built to help you think, not to think for you.

2. Repo tour

Here is every folder and file you will touch, and who owns each one. "You edit this" means you type into it by hand. "The tooling owns this" means a script writes it and you leave it alone.

The 18 category folders (arrays-hashing, two-pointers, sliding-window, stack, binary-search, linked-list, trees, tries, heap-priority-queue, backtracking, graphs, advanced-graphs, one-d-dp, two-d-dp, greedy, intervals, math-geometry and bit-manipulation) are where your solution files go, one file per problem. You edit these. Each starts empty apart from a hidden `.gitkeep` file, which exists only so an empty folder can be saved in git. You can ignore `.gitkeep`.

`redo/` holds fresh re-attempts of problems you solved before. When you redo a problem from scratch, its new file goes here so it does not overwrite your original. You edit these files too. The honesty check does not run on this folder, so a rough redo will not block anything.

`templates/problem-template.py` is the blank solution file that every new problem is copied from. It holds the notes section (called a docstring) and an empty place for your code. You do not edit the template directly. You copy it with a script (see section 4).

`helpers/structures.py` gives you small building blocks for writing tests: `ListNode` and `TreeNode` classes plus functions that turn a plain Python list into a linked list or a tree and back again. These save you from hand building test data. You use these in your tests but you do not need to change them. `helpers/__init__.py` just marks the folder as an importable package.

`scripts/` holds the tooling. The five files you run directly are `new_problem.py`, `progress.py`, `check_file.py`, `flashcards.py` and `install_hooks.py`. Inside `scripts/` there is also a `lib/` folder holding

shared internal code (`neetcode150.py`, the canonical problem list, and `docmeta.py`, the docstring parser). You never run the `lib/` files directly. They are imported by the five scripts.

`hooks/pre-commit` is a safety script that git runs automatically every time you commit.

`scripts/install_hooks.py` puts it in the right place. You run the installer once and then forget about it.

`.github/workflows/ci.yml` tells GitHub to re-run all your solution files every time you push. This is your backstop: if something passes on your machine but is broken, the push will show a red mark. You do not edit this.

`README.md` is the front page. Its top half you can read for a quick summary. Its Progress and Revisit sections are generated by a script, so you never edit those by hand. `python-gotchas.md` is a running list of Python language traps (not algorithm notes) that you add to as you hit them. `CLAUDE.md` holds the rules for the assistant.

`docs/GUIDE.md` is this guide.

The `.claude/commands/` folder holds the slash commands (`/new`, `/hint`, `/review`, `/done`, `/redo`, `/quiz` and `/gotcha`). These are short instruction files that tell the assistant how to run each workflow. You use the commands but rarely need to open these files.

3. Where to work

Do this: read the problem statement on the NeetCode or LeetCode website, then write and test your solution locally in this repo using VS Code, then paste your finished code into LeetCode to submit once your local tests pass.

Each step lives where it does for a reason.

The problem statement and the official judge live on the website. LeetCode owns the exact wording, the examples, the constraints and the hidden test cases that decide whether your answer is truly correct. You cannot reproduce the official judge locally, so you read and submit there.

The thinking, the notes, the tests and the history live here. This repo is where you write the code, record what pattern you used, save the moment you got stuck and keep a permanent record of every problem you have finished. It is also where the honesty check and the spaced repetition schedule run. None of that exists on the website.

So the flow is: understand the problem on the site, solve and test it here, then copy your solution back to the site to get the official pass. Working this way means your local file is the thing you learn from and your LeetCode submission is just the final confirmation.

A note on the local run. You do not need an internet connection to test your code here. Your solution file has its own tests at the bottom (asserts you write), and you run the file to check them. Passing your own tests is not the same as passing LeetCode, because you cannot think of every case the judge will, but it catches most mistakes before you submit.

4. The tools

This section covers each script, each slash command, the pre-commit hook and CI.

A slash command is something you type to the assistant in chat, starting with a slash, for example `/new`. It is a saved instruction that makes the assistant run a workflow for you. A script is a Python program you run in the terminal, for example `python3 scripts/progress.py`. The slash commands mostly wrap the scripts and add the human parts (asking you questions, giving hints).

Run every script from the top of the repo (the folder that contains `README.md`) so that paths line up.

Script: `new_problem.py`

What it does: copies the blank template into a category folder as a new file, named with the zero padded problem number and the slug, and fills in the problem number, the link, the category and the difficulty for you.

Command: `python3 scripts/new_problem.py <category> <number> <slug>`

Example: `python3 scripts/new_problem.py arrays-hashing 1 two-sum`

Expected output: a line confirming the file was created, for example `Created arrays-hashing/0001-two-sum.py`, then your time limit (25 minutes for Easy, 40 minutes for Medium or Hard) and a reminder to read the problem yourself. It refuses to overwrite a file that already exists.

Script: `progress.py`

What it does: looks at every category folder, decides which problems are solved and rewrites the Progress and Revisit sections of the README. A problem counts as solved only when its file has every note filled in and the file runs cleanly.

Command: `python3 scripts/progress.py`

Add `--check` to verify the README is up to date without changing it. This is what CI uses. If it prints that the README is out of sync, run the plain version to fix it.

Expected output: a short summary in the terminal showing how many problems are solved in each category, a total and the current revisit queue. The README file is updated in place.

Script: `check_file.py`

What it does: this is the honesty check. It validates one solution file. It confirms every note field is filled in (none left as TODO), that the hint count is a whole number from 0 to 3, that the date is a real date, that the revisit field is yes or no and that the file runs with all its asserts passing.

Command: `python3 scripts/check_file.py <path>`

Example: `python3 scripts/check_file.py arrays-hashing/0001-two-sum.py`

Expected output: `PASS: <path>` and an exit that means success, or `FAIL: <path>` followed by a bullet list of exactly what is wrong. For example a bullet might read `field 'Pattern' is still TODO/empty`.

Script: flashcards.py

What it does: exports your solved problems to a CSV file you can import into Anki (a flashcard app). The front of each card names the problem and asks which pattern applies. The back shows the pattern and your key insight, taken from your own notes.

Command: `python3 scripts/flashcards.py`

By default it writes `flashcards.csv`. Use `--out cards.csv` for a different name or `--out -` to print to the screen.

Expected output: a line saying how many cards were written and to which file. Only solved problems produce cards.

Script: install_hooks.py

What it does: installs the pre-commit hook into git so the honesty check runs automatically on every commit. You run this once, right after you first clone or open the repo.

Command: `python3 scripts/install_hooks.py`

Expected output: a line confirming the hook was installed.

The slash commands

`/new <category> <number> <slug>` runs the scaffolder, confirms the file, tells you your time limit and says nothing about how to solve the problem.

`/hint <file>` gives you the next hint, one level at a time, and never the algorithm. See section 9 for how the levels work.

`/review <file>` checks the complexity you wrote in your notes against your code, adds three edge case tests to your file, runs it and reports any failures as questions rather than corrections. It never edits your solution.

`/done <file>` walks you through filling any missing notes, sets today's date, runs the honesty check, updates the README and commits that one file.

`/redo` picks the oldest problem due for revisit, restates only the problem (no approach), has you solve it fresh in redo/, then compares your new attempt to the original and talks through what changed.

`/quiz [category]` asks you five short pattern recognition questions drawn only from problems you have already solved, revealing each answer after you try.

`/gotcha <description>` appends a Python language note to `python-gotchas.md`.

The pre-commit hook

Git can run a script right before it saves a commit. This repo uses that to run the honesty check on any solution file you are about to commit. If a file is not finished (a note left as TODO, or a failing test), the commit is stopped.

When it blocks a commit, you see something like this:

```
FAIL: arrays-hashing/0001-two-sum.py
- field 'Pattern' is still TODO/empty

pre-commit: one or more solution files failed the honesty gate.
Fill in the docstring fields and make the asserts pass, then re-commit.
(Emergency bypass, discouraged: git commit --no-verify)
```

The commit does not happen. Fix the listed problems and commit again. The hook only checks files inside category folders, so it never blocks tooling, notes or redo files.

CI

CI (continuous integration) is a check that runs on GitHub, not on your machine. Every time you push, GitHub runs all your solution files and confirms the README is up to date. If anything fails, you get a red mark on the push. It is the same honesty idea as the hook, run again in a clean place so a local quirk cannot hide a broken file.

5. The daily loop

Here is one full problem session, start to finish, with the exact commands.

First, pick the next unsolved problem. Open the README, find the next empty checkbox in the category you are working through and note its number and slug. Open that problem on the LeetCode website and read the statement.

Second, scaffold the file. In chat, type:

```
/new arrays-hashing 1 two-sum
```

The assistant creates `arrays-hashing/0001-two-sum.py` and tells you your time limit. For an Easy problem that is 25 minutes. For Medium or Hard it is 40 minutes. Start a timer now.

Third, open the new file in VS Code. The top is a notes block. The bottom is where you write your solution and your tests. The code area looks like this, with nothing solved yet:

```
class Solution:
    pass # write the method signature from LeetCode yourself

if __name__ == "__main__":
    # write asserts that call your Solution and check the result
    pass
```

Fourth, solve it by hand. Write the method and fill the bottom with your own tests. A test is an assert that calls your method and checks the answer, for example:

```
if __name__ == "__main__":
    sol = Solution()
    # assert sol.your_method(example_input) == expected_output
```


Run the file from the top of the repo to check your tests:

```
python3 arrays-hashing/0001-two-sum.py
```

If it prints nothing and exits, your asserts passed. If an assert fails, Python shows you which line.

Fifth, if you get stuck, ask for a hint:

```
/hint arrays-hashing/0001-two-sum.py
```

You get one hint level. Write down how many hint levels you used. You will record that number in your notes.

Sixth, when your tests pass, get a review:

```
/review arrays-hashing/0001-two-sum.py
```

The assistant adds three edge case tests and checks your complexity claims. If an edge case fails, it asks you a question about it rather than fixing it.

Seventh, finish up:

```
/done arrays-hashing/0001-two-sum.py
```

The assistant asks you for each note (pattern, key insight, where you got stuck, hints used, time and space complexity, whether to revisit), sets today's date, runs the honesty check, updates the README and commits the file.

Last, paste your solution into LeetCode and submit to get the official pass.

What if the timer runs out. Stop coding when the limit hits. Do not keep grinding. Write down in your notes where you got stuck, mark the problem to revisit, and if you want, look at the idea (not the code) through hints. A problem you could not finish in time is exactly the kind you want in your revisit queue. Running out of time is data, not failure.

6. The weekly loop

The daily loop moves you forward. The weekly loop makes what you learned stick.

Spaced repetition is the real mechanism of learning here. You do not remember a pattern because you solved it once. You remember it because you met it again just as you were about to forget it, and had to pull it back. Revisiting on a schedule is what turns a one time solve into a pattern you own. That is why this repo tracks a revisit queue instead of just a checklist.

/redo is the tool for this. Run it a couple of times a week. It picks the oldest problem due in your revisit queue and restates only the problem, with no hint about how you solved it before. You solve it again from a blank file in the redo/ folder. Then the assistant compares your new attempt with the original and you talk through what changed: faster, cleaner, a different bug, a pattern that now feels obvious. If the redo went smoothly, your

original file is marked as no longer needing revisit and its date is refreshed.

`/quiz [category]` is lighter. Run it when you finish a category. It asks five quick questions drawn only from problems you have already solved, so it is pure recall. It reveals each answer after you attempt it. This keeps the patterns warm between full redos.

How the revisit due date is computed. When you finish a problem, the tooling sets a date for when you should see it again. If you needed a hint, or you marked the problem to revisit, that date is 3 days later. If you solved it clean with no hints and did not flag it, the date is 14 days later. Every time you run `progress.py`, any problem whose due date is today or in the past shows up in the Revisit section of the README, oldest first. The logic is simple: problems you found hard come back soon, problems you found easy come back later.

7. How to approach any problem

This is a method you can run on any problem, before you write a line of code. It is the same order every time. With practice it becomes automatic.

Read the problem twice, then restate it in your own words. Do not start coding from a half understanding. Say out loud, or write down, what the input is, what the output is and what makes an answer correct. If you cannot restate it simply, you do not understand it yet.

Work two small examples by hand. Take the sample input and produce the output yourself, on paper, without code. Then make up a second, tiny example and do the same. This is where you notice the real shape of the problem and the edge cases the statement hid.

State the brute force and its cost. Say the most obvious slow solution out loud, even if it is "try every pair". Name its time complexity. This gives you a baseline that any real solution must beat, and sometimes the brute force is fine.

Find the bottleneck, then the structure that removes it. Ask: which step is doing the most repeated work. Then ask: what data structure or ordering would make that step cheap. Repeated membership checks want a hash set. Repeated "smallest so far" wants a heap. Sorted data invites two pointers or binary search. This one question, what removes the repeated work, is most of algorithm design.

Only now, write the code. If you did the steps above, the code is a transcription of a plan you already have, not a search for one.

Test the edge cases. Before you trust it, run it on the empty input, a single element, all duplicates, negative numbers and the largest allowed size. Most bugs live in these corners, not in the normal case.

State the final time and space complexity. Say what your solution costs and be honest about it. If it is no better than the brute force, go back to the bottleneck question. Knowing the cost is part of the answer, not an extra.

8. Category primers

One primer per category, in repo order. Each explains the idea from scratch, tells you the signals that suggest it, lists the key operations and their cost, names the classic beginner mistakes and gives general advice. These teach patterns in the abstract. They do not tell you which problem uses which pattern. Working that out is the exercise.

Arrays and Hashing

An array stores items in a row in memory, so reading the item at a known position is instant, a constant time operation written $O(1)$. A hash table (a dict or set in Python) stores items by running a key through a hash function that decides where to put it, so on average inserting, looking up and deleting a key are all $O(1)$. That average speed is the whole reason hash tables matter: they turn "have I seen this before" from a slow scan into an instant check.

Signals: you find yourself scanning the same data again and again, you need to count how often things appear, you need fast membership tests or you need to remove duplicates.

Key operations: `x in a_set` and `a_dict[key]` are $O(1)$ on average, building a frequency count of `n` items is $O(n)$ and the same membership test on a `list` is a slow $O(n)$.

Classic mistakes: using a list for repeated membership tests (each check walks the whole list), assuming the worst case is always $O(1)$ (heavy collisions can make it $O(n)$) and changing a dict or set while looping over it (Python raises an error).

Advice: the moment you catch yourself scanning a second time, ask whether a set or dict removes the scan.

Two Pointers

Two pointers means walking through a sequence with two positions instead of one. They often start at opposite ends and move toward each other, or start together and move at different speeds. The trick works because moving a pointer lets you rule out many possibilities at once, usually on sorted data.

Signals: the data is sorted and you are looking for a pair or a triple that meets a condition, you are comparing the front against the back or you need to rearrange a sequence in place.

Key operations: a single sweep is $O(n)$ time and the pointers themselves use $O(1)$ extra space.

Classic mistakes: an off-by-one error on the stopping condition, moving the wrong pointer after a comparison and forgetting that many two pointer methods need the data sorted first.

Advice: draw the two positions on paper and step them by hand on a tiny example before coding.

Sliding Window

A sliding window is a range inside a sequence, marked by a left and a right edge, that grows and shrinks as you scan. Instead of re-examining a whole range every time, you update a running summary as items enter on the right and leave on the left. This turns many "best contiguous stretch" questions from slow to fast.

Signals: the problem asks for the longest or shortest run of adjacent items with some property, and the items must be next to each other (contiguous).

Key operations: the whole scan is $O(n)$ because each item enters the window once and leaves once, so the total work is proportional to the length, not the square of it.

Classic mistakes: recomputing the window from scratch each step (which quietly makes it $O(n^2)$), shrinking the window with the wrong condition and recording the answer at the wrong moment.

Advice: keep the window's state (a sum, a count, a set) updated as the edges move, never recomputed.

Stack

A stack is a pile where you add and remove only from the top: last in, first out. Think of a stack of plates. Adding (push), removing (pop) and looking at the top (peek) are all $O(1)$. Stacks shine when the most recent unfinished thing is the one you need next, such as matching opening and closing brackets.

Signals: matching or nesting (brackets, tags), needing the most recent item, undo behaviour or scanning while keeping a running "so far" list that you pop from.

Key operations: push, pop and peek are each $O(1)$.

Classic mistakes: popping when the stack is empty, forgetting to handle items still left on the stack at the end and reaching for a stack when order does not actually matter.

Advice: decide clearly what you push and what event makes you pop.

Binary Search

Binary search finds a value in sorted data by cutting the search range in half each step. Check the middle, then throw away the half that cannot contain the answer. Because you halve the range every time, a range of a million items takes about twenty steps, a cost written $O(\log n)$. The same halving idea also works when you are searching for the smallest answer that satisfies some yes or no test.

Signals: the data is sorted, or you can phrase the goal as "find the smallest value for which this test passes", and you want a fast, roughly logarithmic search.

Key operations: the search is $O(\log n)$ time and $O(1)$ space.

Classic mistakes: an endless loop from updating the bounds wrong, an off-by-one on the low and high edges and confusion over whether an edge is included or excluded.

Advice: pick your rule (which side is inclusive) and keep it consistent through every branch.

Linked List

A linked list is a chain of small boxes (nodes), where each box holds a value and a pointer to the next box. Unlike an array, there is no jumping straight to the tenth item: you follow the chain from the start, which is $O(n)$. But if you already hold a node, inserting or removing next to it is $O(1)$, just a pointer rewrite.

Signals: you are rearranging links rather than values, there is no need for random access, you must reverse or reorder in place or you need to detect a loop.

Key operations: access by position is $O(n)$, insert or delete given the node is $O(1)$.

Classic mistakes: overwriting a node's next pointer before you saved the rest of the chain (losing everything after it), mishandling the empty list or the last node and not using a dummy head node to simplify edge cases.

Advice: use a dummy node at the front and draw every pointer before you move it.

Trees

A tree is nodes connected in a hierarchy: one root at the top, each node holding children below it. A binary tree gives each node at most two children, a left and a right. You visit the nodes either depth first (go deep down one branch before backing up) or breadth first, also called level order (visit everything one level at a time). Recursion fits trees naturally because each child is itself the root of a smaller tree.

Signals: the data is hierarchical, you need to do something at every node or you care about paths from the root downward.

Key operations: visiting every node is $O(n)$, and the depth of the tree sets how deep your recursion goes.

Classic mistakes: forgetting the base case for an empty node, mixing up the visit orders (pre, in and post order) and blowing the recursion stack on a tall, skewed tree.

Advice: decide exactly what your recursive function returns for a single node, then trust it for the children.

Tries

A trie (say "try") is a tree built from the characters of words. Each node stands for a letter, and following a path from the root spells out a prefix. Words that share a prefix share the same early nodes, so a trie is a compact way to store many strings and to answer prefix questions fast. Looking up or inserting a word costs $O(L)$, where L is the length of the word, independent of how many words you have stored.

Signals: you are asked about prefixes, autocomplete style lookups or you have many strings that share leading characters.

Key operations: insert and search are $O(L)$ in the length of the word.

Classic mistakes: forgetting a marker that says "a word ends here" (so you cannot tell a full word from a mere prefix), underestimating the memory a trie uses and rebuilding child maps instead of reusing them.

Advice: give each node two things, a map of child letters and a flag for whether a word ends there.

Heap / Priority Queue

A heap is a structure that always gives you the smallest (or largest) item fast. Python's `heapq` is a min heap: peeking at the smallest is $O(1)$, while pushing a new item or popping the smallest is $O(\log n)$. A priority queue is the general idea a heap implements: a line where the most important item leaves first, regardless of arrival order.

Signals: you repeatedly need the largest or smallest item, you want the "top k" of something, you are merging ordered streams or you are scheduling by priority.

Key operations: peek is $O(1)$, push and pop are $O(\log n)$ each.

Classic mistakes: forgetting Python's `heapq` only does minimums (store negatives to fake a max heap), sorting the whole thing when you only needed the top few and pushing tuples whose later element cannot be compared when the first ties.

Advice: remember `heapq` is min only, and reach for a heap when you want repeated extremes without a full sort.

Backtracking

Backtracking builds an answer one choice at a time and abandons a partial answer the moment it cannot possibly work, then undoes the last choice and tries another. It walks a tree of decisions depth first. It is how you generate every arrangement that fits some rule. Because it can explore many branches, the cost is often exponential, but cutting off dead branches early (pruning) keeps it practical.

Signals: the problem asks for all combinations, all permutations, all subsets or every arrangement that satisfies a constraint.

Key operations: exponential in the worst case, so pruning impossible branches early is what makes it usable.

Classic mistakes: not undoing a change after exploring a branch (so state leaks between attempts), storing a reference to a list that keeps changing instead of a copy and skipping the pruning that would have saved most of the work.

Advice: follow the rhythm choose, explore and un-choose, undoing every change on the way back up.

Graphs

A graph is nodes joined by edges: a general web of connections, of which trees are a special case. You usually store it as an adjacency list, a map from each node to its neighbours. You explore it breadth first (spreading out level by level, which finds the fewest edges to reach a node) or depth first (following one path as far as it goes). A grid of cells is often just a graph in disguise, where each cell connects to its neighbours.

Signals: networks, connectivity, reachability, dependencies or a grid you can treat as connected cells.

Key operations: both breadth first and depth first search cost $O(V + E)$, the number of nodes plus the number of edges.

Classic mistakes: forgetting a visited set and looping forever, treating a directed graph as if edges went both ways and building the neighbour map wrong.

Advice: choose your representation first, and always mark nodes visited as you go.

Advanced Graphs

These are the named graph algorithms for harder questions. Shortest path with weighted edges uses Dijkstra's algorithm, which repeatedly expands the closest unfinished node using a heap. A minimum spanning tree (the cheapest set of edges that connects everything) uses Prim's or Kruskal's algorithm. Ordering tasks with dependencies uses topological sort. Tracking which nodes are joined into groups uses a union-find structure, which answers "are these connected" in almost constant time.

Signals: edges carry weights and you want the cheapest route, you need the minimum wiring to connect everything, you must order steps that depend on each other or you keep merging groups and asking what is connected.

Key operations: Dijkstra is about $O(E \log V)$, union-find operations are nearly $O(1)$ each when done well.

Classic mistakes: running Dijkstra when some edge weights are negative (it does not handle that), skipping the priority queue that makes Dijkstra fast and writing union-find without the path shortening that keeps it quick.

Advice: match the algorithm to the question and to whether edges have weights.

1-D Dynamic Programming

Dynamic programming (DP) solves a big problem by solving smaller versions of it and saving each answer so it is never computed twice. It applies when the same subproblems keep reappearing and the best overall answer is built from best answers to the parts. In one dimensional DP, each subproblem is labelled by a single number, so you can store the answers in a simple list where entry i is the answer for size i .

Signals: the problem asks for a count of ways, or a minimum or maximum cost, and you can see a choice at each step that leads to a smaller version of the same problem.

Key operations: cost is the number of subproblems times the work each one does, often $O(n)$.

Classic mistakes: getting the starting values (base cases) wrong, filling the table in an order where an entry needs one not computed yet and sizing the array off by one.

Advice: write, in a plain sentence, what entry i of your table means before you write any code.

2-D Dynamic Programming

This is the same idea as one dimensional DP, but each subproblem now needs two numbers to describe it, so the answers live in a grid (a table with rows and columns). That happens when you are comparing two sequences at once, moving through a grid or tracking a range with a start and an end. You fill the table cell by cell, each cell built from cells you already filled.

Signals: two sequences compared together, paths across a grid or a subproblem that needs both a start and an end index.

Key operations: cost is $O(n \text{ times } m)$ for a table of n by m cells.

Classic mistakes: getting the table dimensions wrong, filling it in an order that needs cells not ready yet and mixing up which index is the row and which is the column.

Advice: draw the table for a tiny input and fill it in by hand before coding the loops.

Greedy

A greedy method makes the choice that looks best right now at every step and never reconsiders, hoping the local best adds up to the global best. It is fast and simple, but it is only correct when the problem has a special property that guarantees local choices lead to the overall best. Often you sort the data first, then walk it making one obvious pick at a time.

Signals: the problem asks to maximise or minimise something, and there is a natural order to process things in, such as by size or by end time.

Key operations: usually $O(n \log n)$, dominated by the sort.

Classic mistakes: assuming greedy works without checking it, sorting by the wrong key and not testing the greedy answer against a slow brute force on small inputs.

Advice: before trusting greedy, run it against a brute force on many tiny random inputs and confirm they always agree.

Intervals

An interval is a range with a start and an end, like a meeting from 2 to 3. Most interval problems are solved by sorting the ranges (by start, or sometimes by end) and then sweeping through them once, comparing each to the previous. Sorting brings ranges that interact next to each other, so a single pass can merge them, count overlaps or find gaps.

Signals: meetings, bookings, calendars, overlapping ranges or merging ranges into fewer.

Key operations: the sort is $O(n \log n)$, the sweep after it is $O(n)$.

Classic mistakes: sorting by the wrong endpoint, getting the overlap test off by one (do touching ranges count as overlapping) and forgetting to sort at all.

Advice: sort first, then decide precisely what counts as an overlap before you write the comparison.

Math and Geometry

This grab bag covers working with numbers and shapes: digits of an integer, modular arithmetic, greatest common divisors, rotating or scanning a matrix and simple coordinate geometry. There is no single structure. The common thread is careful handling of arithmetic and indices, where a small slip causes a wrong answer.

Signals: the problem is about the digits of a number, arithmetic that might overflow in other languages, remainders or moving around a grid by position.

Key operations: vary by task, so reason about each one directly.

Classic mistakes: integer overflow (rare in Python, which grows integers automatically, but a habit worth keeping), off-by-one errors when indexing a matrix and comparing floating point numbers with exact equality.

Advice: work a small example by hand and watch the boundaries, the first row, the last column, the wraparound.

Bit Manipulation

Every integer is stored as a row of bits (0s and 1s), and you can operate on those bits directly with AND, OR, XOR, NOT and shifts. These operations are fast and sometimes let you solve a problem with no extra memory. XOR is the workhorse: a value XORed with itself is 0 and a value XORed with 0 is unchanged, which lets pairs cancel out. A single integer can also stand in for a set, one bit per possible member (a bitmask).

Signals: the problem hints at doing it without extra space, asks about parity (odd or even counts), involves small sets you could pack into bits or smells like an XOR cancellation.

Key operations: each bit operation is $O(1)$, and scanning all the bits of a number is $O(\text{number of bits})$.

Classic mistakes: tripping over the sign bit on negative numbers, getting the precedence of bit operators wrong (wrap them in parentheses) and confusing the bitwise & with the logical and.

Advice: keep the XOR facts in mind ($x \text{ XOR } x$ is 0, $x \text{ XOR } 0$ is x), they solve more than they first appear to.

9. Getting unstuck

When you are stuck, you ask for a hint with `/hint <file>`. Hints come in levels, and you get one level at a time, in order. You never skip ahead to a bigger hint.

Level 1 is a clarifying question. It nudges your thinking without pointing anywhere, for example asking what the slow version would cost or what you are scanning more than once.

Level 2 points at the relevant line or names a general concept. It might tell you to think about what you could precompute, but it names an idea, not the algorithm.

Level 3 describes the idea in words, with no code and no pseudocode. It is the most you will get, and it still leaves you to write every line.

If you are still stuck after level 3, the help continues as more questions, not as a solution. The assistant will not cross the line into writing your answer.

You must record how many hint levels you used, as a number from 0 to 3, in your notes. This is not bookkeeping for its own sake. The hint count feeds the revisit schedule: any problem where you used a hint comes back to you sooner, because needing a hint means you have not fully learned it yet. Being honest about hints is how the spaced repetition stays useful.

There is an important split in what help is rationed. Help with the algorithm, the actual idea of the solution, is rationed through the hint levels, because finding that idea is the exercise. Help with the Python language is free and unlimited. If you hit a syntax error, a scoping surprise, a shadowed builtin or any quirk of the language itself, ask directly and you get a full explanation. Those traps are not the thing you are practising, so there is no reason to ration them. When one bites you, add it to `python-gotchas.md` with `/gotcha` so you recognise it next time.

10. Troubleshooting

The hook blocked my commit. The pre-commit hook found a solution file that is not finished. Read the bullet list it printed. It names each unfilled note or failing test. Fill in the notes and make your asserts pass, then commit again. Do not reach for `--no-verify` to skip the check; that defeats the point.

Imports fail when I run a file. If you see an error about not finding helpers, you are almost certainly running the file from the wrong folder. Run solution files from the top of the repo, the folder that holds `README.md`, like this: `python3 arrays-hashing/0001-two-sum.py`. The file adds the repo root to its search path on the assumption that you run it from there.

`progress.py` says the `README` is stale. The generated Progress or Revisit section no longer matches your files. This is normal after you finish a problem. Just run `python3 scripts/progress.py` (without `--check`) and it rewrites the sections. Commit the updated `README`.

`check_file` lists `TODO` fields. Your notes are not all filled in. Every field in the top docstring must have a real value, none left as the word `TODO`. The hint count must be a number from 0 to 3, the date must be a real date in year, month, day form and the revisit field must be yes or no. Fill the listed fields and run the check again. The easiest way to fill them correctly is to run `/done`, which asks you for each one.

A solution passes locally but fails on LeetCode. Your own tests passed, but the official judge found a case you did not. This is expected sometimes, because you cannot think of every case the judge tries. Read which case failed on LeetCode, add that case as a new assert in your local file, watch it fail there too, then fix your code until your local test passes again. Now you have both a fixed solution and a permanent test for the case that caught you.